

Overview of Reinforcement learning human feedback

Subject: Reinforcement learning human feedback

1.

Applications RLHF has been applied to various domains of natural language processing (NLP), such as conversational agents, text summarization, and natural language understanding. Ordinary reinforcement learning, in which agents learn from their actions based on a predefined "reward function", is difficult to apply to NLP tasks because the rewards tend to be difficult to define or measure, especially when dealing with complex tasks that involve human values or preferences. RLHF can steer NLP models, in particular language models, to provide answers that align with human preferences with regard to such tasks by capturing their preferences beforehand in the reward model. This results in a model capable of generating more relevant responses and rejecting inappropriate or irrelevant queries. Some notable examples of RLHF-trained language models are OpenAI's ChatGPT (and its predecessor InstructGPT), DeepMind's Sparrow, Google's Gemini, and Anthropic's Claude. In computer vision, RLHF has also been used to align text-to-image models. Studies that successfully used RLHF for this goal have noted that the use of KL regularization in RLHF, which aims to prevent the learned policy from straying too far from the unaligned model, helped to stabilize the training process by reducing overfitting to the reward model. The final image outputs from models trained with KL regularization were noted to be of significantly higher quality than those trained without. Other methods tried to incorporate the feedback through more direct training—based on maximizing the reward without the use of reinforcement learning—but conceded that an RLHF-based approach would likely perform better due to the online sample generation used in RLHF during updates as well as the aforementioned KL regularization over the prior model, which mitigates overfitting to the reward function. RLHF was initially applied to other areas, such as the development of video game bots and tasks in simulated robotics. For example, OpenAI and DeepMind trained agents to play Atari games based on human preferences. In classical RL-based training of such bots, the reward function is simply correlated to how well the agent is performing in the game, usually using metrics like the in-game score. In comparison, in RLHF, a human is periodically presented with two clips of the agent's behavior in the game and must decide which one looks better. This approach can teach agents to perform at a competitive level without ever having access to their score. In fact, it was shown that RLHF can sometimes lead to superior performance over RL with score metrics because the human's preferences can contain more useful information than performance-based metrics. The agents achieved strong performance in many of the environments tested, often surpassing human performance.

2.

Here, β controls how "risk-averse" the value function is (larger β = faster saturation in the logistic function σ) and $z_0 = \text{KL}(\pi_\theta(y' \blacksquare x) \parallel \pi_{\text{ref}}(y' \blacksquare x))$ is a baseline given by the Kullback–Leibler divergence. Since many real-world feedback pipelines yield "like/dislike" data more easily than pairwise comparisons, KTO is designed to be data-efficient and to reflect "loss aversion" more directly by using a straightforward notion of "good vs. bad" at the example level.

3.

Kahneman-Tversky optimization Kahneman-Tversky optimization (KTO) is another direct alignment algorithm drawing from prospect theory to model uncertainty in human decisions. Unlike DPO, KTO requires only a binary feedback signal (desirable or undesirable) instead of explicit preference pairs. The value function $v(x, y)$ is defined piecewise depending on whether y is desirable (λ_D) or undesirable (λ_U):

4.

Identity preference optimization Identity preference optimization (IPO) is a modification to the original DPO objective that introduces a regularization term to reduce the chance of overfitting even when preference data is noisy. To solve this objective, IPO minimizes the quadratic loss function $E_{x, y, w, y_l \sim D} [\mathbb{E}_{\pi}(x, y_w, y_l) - \frac{1}{2} \beta - 1]^2$ where $\mathbb{E}_{\pi}(x, y_w, y_l) = \log \left(\frac{\pi_{\theta}(y_w | x) \pi_{\text{ref}}(y_l | x)}{\pi_{\theta}(y_l | x) \pi_{\text{ref}}(y_w | x)} \right) - \log \left(\frac{\pi_{\theta}(y_l | x) \pi_{\text{ref}}(y_l | x)}{\pi_{\theta}(y_w | x) \pi_{\text{ref}}(y_w | x)} \right)$. IPO can control the gap between the log-likelihood ratios of the policy model and the reference by always regularizing the solution towards the reference model. It allows learning directly from preferences without a reward modelling stage and without relying on the Bradley-Terry modelling assumption that assumes that pairwise preferences can be substituted with pointwise rewards.

5.

Training In RLHF, two different models are trained: a reward model and a reinforcement learning policy. The reward model learns to determine what behavior is desirable based on human feedback, while the policy is guided by the reward model to determine the agent's actions. Both models are commonly initialized using a pre-trained autoregressive language model. This model is then customarily trained in a supervised manner on a relatively small dataset of pairs of prompts to an assistant and their accompanying responses, written by human annotators.

6.

In machine learning, reinforcement learning from human feedback (RLHF) is a technique to align an intelligent agent with human preferences. It involves training a reward model to represent preferences, which can then be used to train other models through reinforcement learning. In classical reinforcement learning, an intelligent agent's goal is to learn a function that guides its behavior, called a policy. The function is iteratively optimized to increase the reward signal derived from the agent's task performance. However, explicitly defining a reward function that accurately approximates human preferences is challenging. Therefore, RLHF seeks to train a "reward model" directly from human feedback. The reward model is first trained in a supervised manner to predict if a response to a given prompt is good (high reward) or bad (low reward) based on ranking data collected from human annotators. This model then serves as a reward function to improve an agent's policy through an optimization algorithm like proximal policy optimization. RLHF has applications in various domains in machine learning, including natural language processing tasks such as text summarization and conversational agents, computer vision tasks like text-to-image models, and the development of video

game bots. While RLHF is an effective method of training models to act better in accordance with human preferences, it also faces challenges due to the way the human preference data is collected. Though RLHF does not require massive amounts of data to improve performance, sourcing high-quality preference data is still an expensive process. Furthermore, if the data is not carefully collected from a representative sample, the resulting model may exhibit unwanted biases.